

AMcoli

Universal MoE Disk-Streaming Inference Engine

Technical Architecture and Codebase Reference Guide

Written by Antigravity AI and Awais

Repository: <https://github.com/Awais-17/AMcoli>

Status: Active Local Development (CMake / C++ / Node.js)

1. Executive Summary (About AMcoli)

AMcoli is a custom-engineered C++ prototyping runtime designed to run massive AI models - specifically Mixture-of-Experts (MoE) models like DeepSeek-V3, GLM-5.2, and Mixtral-8x22B - on standard consumer laptops. Typically, running a model of this magnitude (e.g. 70B to 2.8T parameters) requires expensive datacenter GPUs with hundreds of gigabytes of dedicated VRAM. AMcoli eliminates this hardware constraint by treating the computer's storage drive (SSD/NVMe), system RAM, and GPU VRAM as a unified, fluid memory hierarchy.

The Chef and Kitchen Analogy

To make this easy to understand for anyone, think of a massive Mixture-of-Experts model as a restaurant with a team of 30 specialized chefs (the "Experts"). Your laptop is like a small kitchen counter (VRAM/RAM). There is only enough space for 2 chefs to work concurrently. Trying to force all 30 chefs into the kitchen crashes the system.

AMcoli resolves this by keeping all chefs waiting at a nearby hotel (the Solid-State Drive / SSD). When a query comes in:

1. AMcoli identifies the 2 specific chefs needed for that request.
2. It quickly brings them from the hotel to the counter (Disk-Streaming).
3. Frequently used chefs are kept in the kitchen (Caching).
4. It looks ahead and calls the next needed chefs before they are required (Prefetching).

This dynamic swapping process enables huge models to run on affordable local hardware.

2. Key System Features

Architecture-Generic Parsing

AMcoli parses generic GGUF files and extracts MoE configs (n_experts, top_k, routed weights) automatically, working across Mixtral, DeepSeek, Qwen-MoE, and GLM models.

Three-Tier Cache Hierarchy

Manages a dynamic VRAM slot cache (GPU) and a system RAM Warm cache. Evaluates LRU (Least Recently Used) and LFU (Least Frequently Used) metrics to determine which expert layers to keep loaded or evict.

Asynchronous SSD Disk Streamer

Uses memory mapping (mmap on UNIX, File Mapping APIs on Windows) to stream weights directly into RAM/VRAM over PCIe, utilizing thread pools to overlap data fetching with computation.

Speculative Prefetching

Inspects router logits at layer L to predict which experts will be activated at layer L+1, launching asynchronous disk reads ahead of time so weights are ready by the time computation hits the next layer.

Hardware-Aware Recommendation

Scans system specs on boot (CPU core counts, system RAM availability, free SSD storage space, and GPU adapter name via Win32 DXGI) to recommend the green-flagged optimal model in the selector table.

Agentic CLI Loop (Claude Code style)

An autonomous terminal chat assistant ('amcoli agent') that has access to local files and system commands. It prompts you for permission via a secure confirmation menu before executing tools.

3. Source Code Walkthrough

The codebase is written in C++17 for the core engine, using CMake as the build orchestrator, and JavaScript (Node.js) for the global wrapper interface and the agent loop. Below is an explanation of the core source files:

llama-disk-streamer.cpp / .h

Handles opening the GGUF model files and memory-mapping (mmap) them directly to virtual memory. This allows the operating system to handle paging and virtual allocations. It houses the thread pool responsible for running 'load_expert_layers_async()', enabling concurrent expert parameter loads.

```
// Memory mapping signature on Windows
HANDLE hFile = CreateFileA(path, GENERIC_READ, ...);
HANDLE hMapping = CreateFileMappingA(hFile, NULL, PAGE_READONLY, ...);
void* data = MapViewOfFile(hMapping, FILE_MAP_READ, ...);
```

llama-moe-cache.cpp / .h

Implements the two-tier LRU/LFU cache system. The cache splits VRAM (highest priority) and RAM into distinct slots. When an expert is requested, the engine checks if it exists in VRAM. If not (cache miss), it checks system RAM, and if still missing, it triggers the disk streamer to pull it from the SSD, subsequently evicting the least recently/frequently used expert slot.

amcoli-downloader.cpp / .h

Responsible for talking to the Hugging Face hub. It contains a static registry of 34 major models (including Llama 3.1/3.2, Gemma 2, Qwen 2.5, Command R+, Kimi, and GLM). It handles parsing the download endpoints, spawning curl commands, and displaying animated percentage progress bars in the console.

amcoli-sys-info.cpp / .h

Utilizes native OS APIs to inspect the hardware: Win32 DXGI factory queries are executed to identify the exact GPU adapter (e.g. RTX 3050 Laptop GPU) and dedicated memory sizes, alongside kernel core counting and registry memory checks to feed the recommendation engine.

npm-wrapper/lib/agent.js

Houses the agentic coding workflow. It acts as an autonomous loop that queries a local LLM (via Ollama or Llama.cpp) and parses special JSON blocks returned by the model to trigger system tools. It defines schemas for four key tools: 'list_dir', 'read_file', 'write_file', and 'run_command'. For safety, it wraps all tool executions with an interactive readline approval box.

```
async function runAgentStep() {
  const response = await queryLLM(chatHistory);
  const toolCall = parseToolCall(response);
  if (toolCall) {
    const approved = await askApproval(toolCall.tool, toolCall.args);
    if (approved) {
      const result = await Tools[toolCall.tool](toolCall.args);
      chatHistory.push({ role: 'user', content: result });
      await runAgentStep(); // Loop back
    }
  }
}
```

4. CLI Command Reference

AMcoli offers multiple execution pathways globally from your terminal. Here is the list of available CLI commands:

1. **amcoli run**

Launches the C++ interactive model selector menu. It auto-detects hardware specs, lists all 34 GGUF models, flags the recommended ones, and lets you choose one to download or stream.

2. **amcoli run --model <path>**

Directly loads and inspects the specified local GGUF file. It parses the GGUF header tensor keys, sets up the SSD streamer mapping, and verifies core model parameters.

3. **amcoli agent**

Launches the autonomous agentic chat assistant in your current terminal directory. The agent can inspect files, list directories, make code changes, and compile projects upon your keyboard confirmation.

4. **amcoli agent --api-url <url> --model <model-name>**

Allows overriding the local LLM endpoint (e.g. connecting to LM Studio or custom Llama.cpp servers instead of the default Ollama server).